

POLYMORPHISM PRACTICE PROBLEMS

PRACTICE PROBLEM 1: Gaming Arena - Method Overloading

Understanding compile-time polymorphism through method overloading in a gaming context.

```
package Week7.Practice;
public class GameBattle {

    public void attack(int damage) {
        System.out.println("Basic attack for " + damage + " points!");
    }

    public void attack(int damage, String weapon) {
        System.out.println("Attacking with " + weapon + " for " + damage + "
points!");
    }

    public void attack(int damage, String weapon, boolean isCritical) {
        if (isCritical) {
            System.out.println("CRITICAL HIT! " + weapon + " deals " + (damage
* 2) + " points!");
        } else {
            attack(damage, weapon);
        }
    }

    public void attack(int damage, String[] teammates) {
        int totalDamage = damage * teammates.length;
        System.out.print("Team attack with ");
        for (int i = 0; i < teammates.length; i++) {
            System.out.print(teammates[i]);
        }
    }
}
```

```

        if (i != teammates.length - 1) System.out.print(", ");
    }
    System.out.println(" for " + totalDamage + " total damage!");
}

public static void main(String[] args) {
    GameBattle battle = new GameBattle();

    battle.attack(50);
    battle.attack(75, "Sword");
    battle.attack(60, "Bow", true);
    battle.attack(40, new String[]{"Alice", "Bob"});
}
}

```

OUTPUT:

```

Basic attack for 50 points!
Attacking with Sword for 75 points!
CRITICAL HIT! Bow deals 120 points!
Team attack with Alice, Bob for 80 total damage!
PS C:\Users\jamwa\OneDrive\Desktop\STEP\Sem3>

```

PRACTICE PROBLEM 2: Social Media Platform - Method Overriding

Demonstrating runtime polymorphism through method overriding in social media context.

```

package Week7.Practice;

class SocialMediaPost {
    protected String content;
    protected String author;

    public SocialMediaPost(String content, String author) {
        this.content = content;
    }
}

```

```

        this.author = author;
    }

    public void share() {
        System.out.println("Sharing: " + content + " by " + author);
    }
}

class InstagramPost extends SocialMediaPost {
    private int likes;

    public InstagramPost(String content, String author, int likes) {
        super(content, author);
        this.likes = likes;
    }

    @Override
    public void share() {
        System.out.println("Instagram: " + content + " by @" + author + " - " +
likes + " likes");
    }
}

class TwitterPost extends SocialMediaPost {
    private int retweets;

    public TwitterPost(String content, String author, int retweets) {
        super(content, author);
        this.retweets = retweets;
    }

    @Override
    public void share() {
        System.out.println("Tweet: " + content + " by @" + author + " - " +

```

```

retweets + " retweets");
    }
}

public class SocialMediaDemo {
    public static void main(String[] args) {
        SocialMediaPost[] feed = new SocialMediaPost[3];

        feed[0] = new InstagramPost("Sunset vibes!", "jamwalvartika296", 245);
        feed[1] = new TwitterPost("Java is awesome!", "biswassoham", 89);
        feed[2] = new SocialMediaPost("Hello world!", "jamwalm");

        for (SocialMediaPost post : feed) {
            post.share();
        }
    }
}

```

OUTPUT:

```

Instagram: Sunset vibes! by @jamwalvartika296 - 245 likes
Tweet: Java is awesome! by @biswassoham - 89 retweets
Sharing: Hello world! by jamwalm
PS C:\Users\jamwa\OneDrive\Desktop\STEP\Sem3>

```

PRACTICE PROBLEM 3: Food Delivery App - Dynamic Method Dispatch

Exploring how JVM resolves method calls at runtime based on actual object type.

```

package Week7.Practice;

class Restaurant {
    protected String name;

    public Restaurant(String name) {
        this.name = name;
    }
}

```

```

        public void prepareFood() {
            System.out.println(name + " is preparing generic food");
        }

        public void estimateTime() {
            System.out.println("Estimated time: 30 minutes");
        }
    }

class PizzaPlace extends Restaurant {
    public PizzaPlace(String name) {
        super(name);
    }

    @Override
    public void prepareFood() {
        System.out.println(name + " is making delicious pizza with fresh
toppings!");
    }

    @Override
    public void estimateTime() {
        System.out.println("Pizza ready in 20 minutes!");
    }
}

class SushiBar extends Restaurant {
    public SushiBar(String name) {
        super(name);
    }

    @Override
    public void prepareFood() {
        System.out.println(name + " is crafting fresh sushi with precision!");
    }

    @Override
    public void estimateTime() {

```

```

        System.out.println("Sushi will be ready in 25 minutes!");
    }
}

public class FoodDelivery {
    public static void main(String[] args) {
        Restaurant restaurant = new PizzaPlace("Mario's Pizza");
        restaurant.prepareFood();
        restaurant.estimateTime();

        restaurant = new SushiBar("Tokyo Sushi");
        restaurant.prepareFood();
        restaurant.estimateTime();
    }
}

```

OUTPUT:

```

Mario's Pizza is making delicious pizza with fresh toppings!
Pizza ready in 20 minutes!
Tokyo Sushi is crafting fresh sushi with precision!
Sushi will be ready in 25 minutes!
PS C:\Users\jamwa\OneDrive\Desktop\STEP\Sem3>

```

PRACTICE PROBLEM 4: University System - Upcasting Adventures

Learning safe upcasting and accessing inherited members in university context.

```

package Week7.Practice;

class Person {
    protected String name;
    protected int age;
    protected String email;

    public Person(String name, int age, String email) {

```

```
        this.name = name;
        this.age = age;
        this.email = email;
    }

    public void introduce() {
        System.out.println("Hi! I'm " + name + ", " + age + " years old.");
    }

    public void getContactInfo() {
        System.out.println("Email: " + email);
    }
}

class Student extends Person {
    private String studentId;
    private String major;

    public Student(String name, int age, String email, String studentId, String
major) {
        super(name, age, email);
        this.studentId = studentId;
        this.major = major;
    }

    public void attendLecture() {
        System.out.println(name + " is attending " + major + " lecture");
    }

    public void submitAssignment() {
        System.out.println("Assignment submitted by " + studentId);
    }
}
```

```

class Professor extends Person {
    private String department;

    public Professor(String name, int age, String email, String department) {
        super(name, age, email);
        this.department = department;
    }

    public void conductClass() {
        System.out.println("Prof. " + name + " is teaching " + department);
    }
}

public class UniversitySystem {
    public static void main(String[] args) {
        Student s = new Student("Vartika", 20, "vj1839@srmist.edu", "CS2021",
"Computer Science");
        Person p = s;
        p.introduce();
        p.getContactInfo();

        System.out.println("Accessing field via Person reference: " + p.name);
    }
}

```

OUTPUT:

```

Hi! I'm Vartika, 20 years old.
Email: vj1839@srmist.edu
Accessing field via Person reference: Vartika
PS C:\Users\jamwa\OneDrive\Desktop\STEP\Sem3>

```

PRACTICE PROBLEM 5: Entertainment System

- Mastering Downcasting

Learning explicit downcasting to access subclass-specific functionality.

```
package Week7.Practice;

class Entertainment {
    protected String title;

    public Entertainment(String title) {
        this.title = title;
    }

    public void start() {
        System.out.println("Starting " + title);
    }

    public void stop() {
        System.out.println("Stopping " + title);
    }
}

class Movie extends Entertainment {
    private String genre;

    public Movie(String title, String genre) {
        super(title);
        this.genre = genre;
    }

    public void showSubtitles() {
        System.out.println("Showing subtitles for " + title + " (" + genre +
            ")");
    }

    public void adjustQuality() {
        System.out.println("Adjusting video quality for " + title);
    }
}
```

```

class Game extends Entertainment {
    private String platform;

    public Game(String title, String platform) {
        super(title);
        this.platform = platform;
    }

    public void saveProgress() {
        System.out.println("Saving " + title + " progress on " + platform);
    }

    public void showLeaderboard() {
        System.out.println(title + " leaderboard on " + platform);
    }
}

public class EntertainmentHub {
    public static void main(String[] args) {
        Entertainment entertainment = new Movie("Avengers", "Action");
        entertainment.start();

        Movie m = (Movie) entertainment;
        m.showSubtitles();
        m.adjustQuality();

        entertainment = new Game("FIFA 24", "PlayStation");
        entertainment.start();

        Game g = (Game) entertainment;
        g.saveProgress();
        g.showLeaderboard();

        // Wrong downcast demonstration
        try {
            Movie wrong = (Movie) entertainment; // entertainment is actually a
Game
            wrong.adjustQuality();
        } catch (ClassCastException e) {

```

```
        System.out.println("Wrong downcast detected: " + e);
    }
}
```

OUTPUT:

```
Starting Avengers
Showing subtitles for Avengers (Action)
Adjusting video quality for Avengers
Starting FIFA 24
Saving FIFA 24 progress on PlayStation
FIFA 24 leaderboard on PlayStation
Wrong downcast detected: java.lang.ClassCastException: class Week7.Practice.Game cannot be cast to class W
Week7.Practice.Movie (Week7.Practice.Game and Week7.Practice.Movie are in unnamed module of loader 'app')
PS C:\Users\jamwa\OneDrive\Desktop\STEP\Sem3>
```